Rozwiązanie: Klon "r/place" (Spring Boot)

Oto kompletne rozwiązanie kolokwium z 2024 roku, tworzące serwer webowy na architekturze Spring Boot [cite: 1, 2]. Aplikacja zarządza tokenami użytkowników (krok 1 i 2) [cite: 3, 5], wystawia obraz płótna (krok 3) [cite: 11] oraz przyjmuje piksle (krok 4) [cite: 13], obsługując bazę danych SQLite (krok 5, 6) [cite: 31, 34]. Zawiera również serwer dla administratora umożliwiający banowanie użytkowników i generowanie wideo `.mp4` [cite: 36, 47, 50]. Przekazane przez Ciebie pliki obrazów `kon_*.png` demonstrują dokładnie ten mechanizm klatkowania, który został wykorzystany w komendzie do ffmpeg [cite: 66, 67, 68].

1. Zależności (pom.xml)

```xml
<!-- Należy dodać do standardowego projektu Spring Boot Web -->
<dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
    <version>3.41.2.1</version>
</dependency>
```

2. Klasy Danych (Rekordy)

```java
package com.example.place;

import java.time.LocalDateTime;

public record RegisterResponse(String token, LocalDateTime timestamp) {}
public record TokenInfo(String token, LocalDateTime generatedAt, boolean isActive) {}
public record PixelRequest(String id, int x, int y, String color) {}
```

3. Kontroler REST (PlaceController.java)

```java
package com.example.place;

import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.io.IOException;
import java.util.List;

@RestController
public class PlaceController {

    private final PlaceService placeService;

    public PlaceController(PlaceService placeService) {
        this.placeService = placeService;
    }

    // Krok 1: Rejestracja
    @PostMapping("/register")
    public RegisterResponse register() {
        return placeService.register();
    }

    // Krok 2: Pobieranie listy tokenów
    @GetMapping("/tokens")
    public List<TokenInfo> getTokens() {
        return placeService.getTokens();
    }

    // Krok 3: Wyświetlanie obrazka
    @GetMapping(value = "/image", produces = MediaType.IMAGE_PNG_VALUE)
    public byte[] getImage() throws IOException {
        return placeService.getImageBytes();
    }

    // Krok 4: Ustawianie piksela
    @PostMapping("/pixel")
    public ResponseEntity<Void> setPixel(@RequestBody PixelRequest request) {
        if (!placeService.isTokenActive(request.id())) {
            // Kod 302 wg instrukcji ("302 Forbidden, jeżeli token nie jest aktywny")
            return new ResponseEntity<>(HttpStatus.FOUND);
        }
        if (request.x() < 0 || request.x() >= 512 || request.y() < 0 || request.y() >= 512) {
            return new ResponseEntity<>(HttpStatus.BAD_REQUEST);
        }

        placeService.setPixel(request);
        return new ResponseEntity<>(HttpStatus.OK);
    }
}
```

4. Logika Biznesowa (PlaceService.java)

```java
package com.example.place;

import jakarta.annotation.PostConstruct;
import org.springframework.stereotype.Service;

import javax.imageio.ImageIO;
import java.awt.image.BufferedImage;
import java.io.*;
import java.net.ServerSocket;
import java.net.Socket;
import java.sql.*;
import java.time.LocalDateTime;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

@Service
public class PlaceService {
    private final Map<String, LocalDateTime> tokens = new ConcurrentHashMap<>();
    private BufferedImage canvas = new BufferedImage(512, 512, BufferedImage.TYPE_INT_RGB);
    private Connection conn;

    @PostConstruct
    public void init() throws Exception {
        conn = DriverManager.getConnection("jdbc:sqlite:place.db");
        try (Statement stmt = conn.createStatement()) {
            stmt.execute("CREATE TABLE IF NOT EXISTS entry (token TEXT NOT NULL, x INTEGER NOT
NULL, y INTEGER NOT NULL, color TEXT NOT NULL, timestamp TEXT NOT NULL);");
        }
        redrawCanvas();
        new Thread(this::startAdminServer).start();
    }

    public RegisterResponse register() {
        String token = UUID.randomUUID().toString();
        LocalDateTime now = LocalDateTime.now();
        tokens.put(token, now);
        return new RegisterResponse(token, now);
    }

    public List<TokenInfo> getTokens() {
        LocalDateTime now = LocalDateTime.now();
        List<TokenInfo> result = new ArrayList<>();
        for (Map.Entry<String, LocalDateTime> entry : tokens.entrySet()) {
            boolean active = entry.getValue().plusMinutes(5).isAfter(now);
            result.add(new TokenInfo(entry.getKey(), entry.getValue(), active));
        }
        return result;
    }

    public boolean isTokenActive(String token) {
        LocalDateTime time = tokens.get(token);
        return time != null && time.plusMinutes(5).isAfter(LocalDateTime.now());
    }

    public byte[] getImageBytes() throws IOException {
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        synchronized (canvas) {
            ImageIO.write(canvas, "png", baos);
        }
        return baos.toByteArray();
    }
```

```
    public void setPixel(PixelRequest req) {
        synchronized (canvas) {
            canvas.setRGB(req.x(), req.y(), Integer.parseInt(req.color(), 16));
        }
        try (PreparedStatement pstmt = conn.prepareStatement("INSERT INTO entry (token, x, y,
color, timestamp) VALUES(?, ?, ?, ?, ?)")) {
            pstmt.setString(1, req.id());
            pstmt.setInt(2, req.x());
            pstmt.setInt(3, req.y());
            pstmt.setString(4, req.color());
            pstmt.setString(5, LocalDateTime.now().toString());
            pstmt.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    private void redrawCanvas() {
        BufferedImage newCanvas = new BufferedImage(512, 512, BufferedImage.TYPE_INT_RGB);
        try (Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery("SELECT token, x, y, color FROM entry ORDER BY
timestamp")) {
            while (rs.next()) {
                int x = rs.getInt("x");
                int y = rs.getInt("y");
                String color = rs.getString("color");
                newCanvas.setRGB(x, y, Integer.parseInt(color, 16));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        synchronized (canvas) {
            canvas = newCanvas;
        }
    }

    // Krok 7: Serwer administratora
    private void startAdminServer() {
        try (ServerSocket serverSocket = new ServerSocket(9000)) {
            while (true) {
                try (Socket socket = serverSocket.accept();
                     BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
                     PrintWriter out = new PrintWriter(socket.getOutputStream(), true)) {

                    out.println("Hasło:");
                    String pass = in.readLine();
                    if (!"admin".equals(pass)) {
                        out.println("Błędne hasło.");
                        continue;
                    }

                    out.println("Komendy: ban <TOKEN>, video");
                    String line;
                    while ((line = in.readLine()) != null) {
                        if (line.startsWith("ban ")) {
                            String tokenToBan = line.substring(4).trim();
                            tokens.remove(tokenToBan); // Unieważnienie
                            try (PreparedStatement pstmt =
conn.prepareStatement("DELETE FROM entry WHERE token=?")) {
                                pstmt.setString(1, tokenToBan);
                                int deleted = pstmt.executeUpdate();
                                out.println("Usunięto rekordów: " + deleted);
```

```
                        }
                        redrawCanvas(); // Krok 8: Regeneracja obrazu
                    } else if (line.equals("video")) {
                        generateVideo(out); // Krok 9: Tworzenie wideo
                    }
                }
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}

private void generateVideo(PrintWriter out) {
    try {
        File workDir = new File("workdir");
        if (!workDir.exists()) workDir.mkdir();

        // Czyszczenie starych ramek (dla bezpieczeństwa)
        for (File f : workDir.listFiles()) if (f.getName().endsWith(".png")) f.delete();

        BufferedImage tempCanvas = new BufferedImage(512, 512, BufferedImage.TYPE_INT_RGB);
        int frameIdx = 0;
        try (Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery("SELECT token, x, y, color FROM entry ORDER BY
timestamp")) {
            while (rs.next()) {
                int x = rs.getInt("x");
                int y = rs.getInt("y");
                tempCanvas.setRGB(x, y, Integer.parseInt(rs.getString("color"), 16));

                File frameFile = new File(workDir, String.format("frame_%05d.png", frameIdx+
+));
                ImageIO.write(tempCanvas, "png", frameFile);
            }
        }

        ProcessBuilder pb = new ProcessBuilder(List.of(
            "ffmpeg", "-y", "-framerate", "30", "-pattern_type", "glob",
            "-i", "workdir/*.png", "-c:v", "libx265", "out.mp4"
        ));
        pb.inheritIO();
        Process p = pb.start();
        p.waitFor();

        out.println("Video wygenerowano pomyslnie (out.mp4).");
    } catch (Exception e) {
        out.println("Blad generowania video: " + e.getMessage());
    }
}
}
```